

Table 1.1 **Topics for one quarter (10 Weeks) of a scientific computing course.** Units are indicated by I, II, and III, and the visualization, here spread out into several laboratory periods, can be completed in one. Options: week 3 on visualization; postpone matrix computing; postpone hardware basics; devote a week to OOP; include hardware basics in week 2.

<i>Week</i>	<i>Topics</i>	<i>Chapter</i>
1	OS tools, limits	1, (4)
2	Errors, visualization	2, 3
3	Monte Carlo, visualization	5, 3
4	Integration, visualization	6, (3)
5	Derivatives, searching	7I, II
6	Matrices, N-D search	8I
7	Data fitting	8II
8	ODE oscillations	9I
9	ODE eigenvalues	9II
10	Hardware basics	14I, III⊙

expressing in words what has been done and what can be concluded. Further exploring is encouraged. Although we recognize that programming is a valuable skill for scientists, we also know that it is incredibly exacting and time-consuming. In order to lighten the workload somewhat, we provide “bare bones” programs in the text and via hyperlinks. We recommend that these be used as guides for the reader’s own programs or tested and extended to solve the problem at hand. As part of this approach we suggest that the learner write up a mini lab report for each problem containing

Equations solved	Numerical method	Code listing
Visualization	Discussion	Critique

The report should be an executive summary of the type given to a boss or manager; make it clear that you understand the materials but do not waste everyone’s time.

One of the most rewarding uses of computers is *visualizing* and analyzing the results of calculations with 2-D and 3-D plots, with color, and with animation. This assists in the debugging process, hastens the development of physical and mathematical intuition, and increases the enjoyment of the work. It is essential that you learn to use visualization tools as soon as possible, and so in Chapter 3, “Visualization Tools,” we describe a number of free visualization tools that we use and recommend. We include many color figures showing visualizations (the printed text, unfortunately, permits only gray scale).

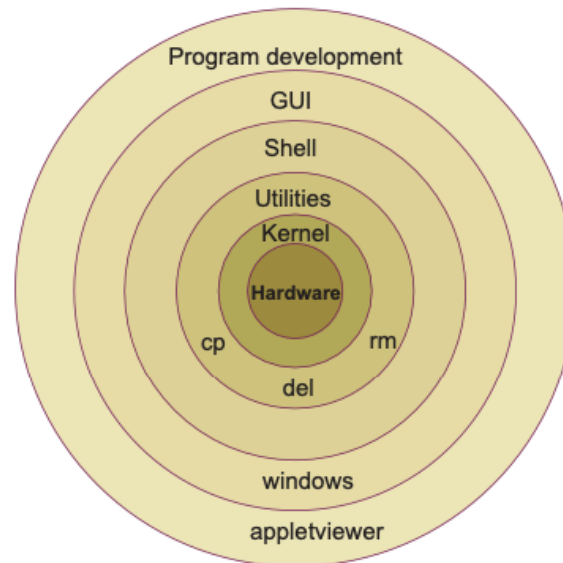
1.5 MAKING COMPUTERS OBEY; LANGUAGES (THEORY)

Computers are incredibly fast, accurate, and stupid; humans are incredibly slow, inaccurate, and brilliant; together they are powerful beyond imagination.

— *Albert Einstein*

 As anthropomorphic as your view of your computer may be, keep in mind that computers always do exactly as they are told. This means that you must tell them exactly everything they have to do. Of course the programs you run may have such convoluted logic that you may not

Figure 1.7 A schematic view of a computer's kernel and shells. The hardware is in the center surrounded by increasing higher-level software.



have the endurance to figure out the details of what you have told the computer to do, but it is always possible in principle. So your first **problem** is to obtain enough understanding so that you feel well enough in control, no matter how illusionary, to figure out what the computer is doing.

Before you tell the computer to obey your orders, you need to understand that life is not simple for computers. The instructions they understand are in a *basic machine language*  ³ that tells the hardware to do things like move a number stored in one memory location to another location or to do some simple binary arithmetic. Very few computational scientists talk to computers in a language computers can understand. When writing and running programs, we usually communicate to the computer through *shells*, in *high-level languages* (Python, Java, Fortran, C), or through *problem-solving environments* (Maple, Mathematica, and Matlab). Eventually these commands or programs are translated into the basic machine language that the hardware understands.

A *shell*  is a *command-line interpreter*, that is, a set of small programs run by a computer that respond to the commands (the names of the programs) that you key in. Usually you open a special window to access the shell, and this window is called a shell as well. It is helpful to think of these shells as the outer layers of the computer's operating system (OS) (Figure 1.5), within which lies a *kernel*  of elementary operations. (The user seldom interacts directly with the kernel, except possibly when installing programs or when building an operating system from scratch.) It is the job of the shell to run programs, compilers, and utilities that do things like copying files. There can be different types of shells on a single computer or multiple copies of the same shell running at the same time.

Operating systems have names such as *Unix*, *Linux*, *DOS*, *MacOS*, and *MS Windows*. The *operating system*  is a group of programs used by the computer to communicate with users and devices, to store and read data, and to execute programs. Under Unix and Linux, the OS tells the computer what to do in an elementary way, while Windows includes various graphical elements as part of the operating system (this increases speed at the cost of complexity). The OS views you, other devices, and programs as input data for it to process;

³The Beginner's All-Purpose Symbolic Instruction Code (BASIC) programming language of the original PCs should not be confused with basic machine language.

in many ways, it is the indispensable office manager. While all this may seem complicated, the purpose of the OS is to let the computer do the nitty-gritty work so that you can think higher-level thoughts and communicate with the computer in something closer to your normal everyday language.

When you submit a program to your computer in a *high-level language* , the computer uses a compiler to process it. The *compiler*  is another program that treats your program as a foreign language and uses a built-in dictionary and set of rules to translate it into basic machine language. As you can probably imagine, the final set of instructions is quite detailed and long and the compiler may make several passes through your program to decipher your logic and translate it into a fast code. The translated statements form an *object*  or compiled code, and when *linked*  together with other needed subprograms, form a load module. A *load module*  is a complete set of machine language instructions that can be *loaded* into the computer's memory and read, understood, and followed by the computer.

Languages such as *Fortran* and *C* use compilers to read your entire program and then translate it into basic machine instructions. Languages such as *BASIC* and *Maple* translate each line of your program as it is entered. Compiled languages usually lead to more efficient programs and permit the use of vast subprogram libraries. Interpreted languages give a more immediate response to the user and thereby appear “friendlier.” The Python and Java languages are a mix of the two. When you first compile your program, Python interprets it into an intermediate, universal *byte code*  which gets stored as a PYC (or PYO) file. This file can be transported to and used on other computers, although not with different versions of Python. Then, when you run your program, Python recompiles the byte code into a machine-specific compiled code, which is faster than starting from source code.

1.6 PROGRAMMING WARMUP

Before we go on to serious work, we want to ensure that your local computer is working right for you. Assume that calculators have not yet been invented and that you need a program to calculate the area of a circle. Rather than use any specific language, write that program in pseudocode that can be converted to your favorite language later. The first program tells the computer:⁴

```
Calculate area of circle                                     # Do this computer!
```

This program cannot really work because it does not tell the computer which circle to consider and what to do with the area. A better program would be

```
read radius                                                # Input
calculate area of circle                                    # Numerics
print area                                                  # Output
```

The instruction **calculate area of circle** has no meaning in most computer languages, so we need to specify an *algorithm*,  that is, a set of rules for the computer to follow:

```
read radius                                                # Input
PI = 3.141593                                              # Set constant
area = PI * r * r                                          # Algorithm
print area                                                  # Output
```

⁴Comments placed in the field to the right are for your information and *not* for the computer to act upon.